Real-Time Embedded Control of an Active Rear Wing under Emulated Aerodynamic Load

Gabriel Chuffart, *276671*, *gabriel.chuffart@studenti.unitn.it*

October 20, 2026

1 Introduction

Active aerodynamic systems trade downforce for drag in real time. Since 2011, Formula 1 cars have used the Drag Reduction System (DRS): an actuator opens the upper flap of the rear wing on the straights to reduce drag, and the flap closes automatically as soon as the driver brakes, when downforce is needed again [1]. Adjustable rear wings and front devices are also studied on road and track cars to shape the front/rear downforce balance [2], and lap-time simulations show that adaptive aerodynamics brings a measurable gain over a static set-up [3].

From an embedded-systems point of view, this system is a hard real-time actuator. There are three main problems suitable for a low-cost implementation. First, the flap must hold its position against an aerodynamic hinge moment that grows with the square of the speed [4]. Second, the braking event must be detected and the flap closed within a given latency. Third, any fault (power loss, sensor failure, software hang) must bring the flap back to the high-downforce position, taking inspiration from the fail-safe principles of ISO 26262 [5].

Testing this kind of system on a real car is neither safe nor repeatable. Actuators are therefore validated on load simulators that reproduce the aerodynamic hinge moment on the actuator shaft [6]. This project applies the same approach at a smaller scale: a 3D-printed flap is driven by a direct-drive brushless motor, while a second motor emulates the aerodynamic load computed from real driving data. The goal is to compare three control architectures and to characterise the real-time behaviour, the fail-safe behaviour and the power consumption of the system.

2 Project Description

The goal is an embedded active-wing controller (the "Wing ECU") that (i) estimates the vehicle state on board, (ii) decides when to open or close the flap, (iii) tracks the flap angle under aerodynamic load, and (iv) reaches the safe position on any fault.

The aerodynamic hinge moment is modelled as

$$M_h = \tfrac{1}{2}\,\rho\, v^2\, S\, c\, C_h(\delta), \tag{1}$$

where v is the vehicle speed, S and c are the flap area and mean chord, and $C_h(\delta)$ is the hinge-moment coefficient as a function of the flap angle δ, taken from the literature. A scaled version of M_h is applied in real time by the load-emulator motor. The bench is a simplified, small-scale model inspired by the F1 DRS: the aim is to study the embedded control problem with student-level hardware, not to reach F1-level performance. The initial sizing is summarised in Table 1.

The core of the project is a comparison of three position-control architectures under identical replayed driving profiles:

C1 PID position control;
C2 PID with feed-forward compensation of the modelled hinge moment;
C3 PID with a disturbance observer that estimates the load torque online [7].

Parameter	Initial value
Scale factor (bench / F1)	1/50
Hinge axis position	0.55 c from the leading edge
Maximum flap opening	40°
Actuator and emulator motors	2 identical GM4108H-120T gimbal motors
Return spring	low-stiffness torsion spring, preloaded towards closed

Table 1: Initial bench sizing (to be refined in M1).

Validation measurements. The initial targets below will be refined after milestone M1. Every measurement is repeated for C1–C3 under the same driving profiles.

Control performance

- Flap tracking error (RMS and peak) and closing time (10–90 %) at several emulated speeds. Target: closing time below 100 ms at the highest emulated load.

Delays and timing

- End-to-end braking delay, split into stages: start of braking → detection on the IMU → close decision → close command → flap at 90 % closed. Each stage is time-stamped by a GPIO toggle and read on the oscilloscope; the flap position comes from the encoder.
- CAN latency between the two boards (message sent → received and processed).
- Latency, jitter and maximum measured execution time of each RTOS task under load, measured with GPIO toggling, the DWT cycle counter and SEGGER SystemView.
- Fault reaction time for injected faults (power cut, encoder disconnected, frozen task, lost CAN messages). Target: flap in the safe position within 200 ms.

Power and energy

- Electrical power of each board, measured by a low-cost current/voltage monitor on the 12 V input and logged with the other signals. The bench power supply is used as a cross-check.
- Energy per open/close cycle, holding power with the flap open, and energy over a full lap profile, for C1–C3.
- Influence of the return-spring stiffness: a stiffer spring closes faster on a fault but costs more holding current when the flap is open.

2.1 Hardware

Two identical custom PCBs are designed in KiCad: one is the Wing ECU, the other drives the load emulator. They communicate over CAN. Most of the components (STM32 microcontrollers and development boards) are provided by the lab; firmware development starts on lab development boards while the custom PCBs are manufactured.

- **MCU:** STM32G474 (Cortex-M4F, 170 MHz, FD-CAN, advanced motor-control timers), available in the lab. A similar MCU could replace it; this one was chosen for its motor-control peripherals.
- **Motor drivers:** one integrated three-phase driver with current sensing per board.
- **Flap actuator:** direct-drive brushless gimbal motor. Direct drive avoids gearbox backlash, and the phase current gives an estimate of the torque.
- **Load emulator:** a second, identical motor coupled to the flap shaft.
- **Position sensing:** 14-bit magnetic encoders on both shafts.
- **Vehicle state:** 6-axis IMU and GNSS module on the Wing ECU, so that the board needs no connection to the car (the OBD port is slow, and most useful data are proprietary to each manufacturer).
- **Power measurement:** low-cost current/voltage monitor (INA226 type) on the 12 V input of each board.
- **Mechanics:** 3D-printed flap, shaft supports and coupling, ball bearings, torsion return spring.

- **Supply:** 12 V bench supply, local 5 V and 3.3 V regulators.

Measurements use the on-board sensors (encoders, phase currents, IMU, power monitors), GPIO toggling with an oscilloscope for timing, and the bench power supply as an energy reference.

2.2 Software

- **OS:** FreeRTOS [8] on the STM32G474.
- **Language and drivers:** C with the STM32Cube LL/HAL drivers and CMSIS-DSP.
- **Motor control:** field-oriented current control written on the LL drivers, with the ST Motor Control SDK as a fallback.
- **Task planning:** current loop in the PWM interrupt; flap position loop; vehicle-state estimation and braking detection; open/close logic and safety monitor; power monitoring and data logging. On the emulator board: the hinge-moment model and the torque loop.
- **Instrumentation:** GPIO time-stamps at each stage of the braking chain, DWT cycle counter for execution times, synchronised logging of position, current and power.
- **Safety:** hardware watchdog, independent supervision task, spring-driven return to the safe position on any loss of control.
- **Off-line tools:** MATLAB for the aerodynamic model, the synthetic lap profiles and the analysis of the measurements (aerodynamic and lap-profile optimisation are outside the scope of this embedded-systems project); SEGGER SystemView for RTOS tracing.

2.3 Milestones and Time Estimation (subject to change)

- **M1** (around 35 h): PCBs assembled; field-oriented control and flap position loop running without load; IMU/GNSS and power monitors acquired; timing instrumentation in place.
- **M2** (around 45 h): two-motor bench with load emulation; C1–C3 implemented and compared on synthetic lap profiles, including tracking, timing and energy.
- **M3** (around 50 h): real driving data replayed on the bench; braking detection and end-to-end delay budget; fault-injection process; power and energy campaign; final measurements and report.

2.4 Stretch goals

- **S1:** earlier braking detection from the brake-light signal, read through an optocoupler without cutting any wire, and comparison of the resulting delay with IMU-based detection.
- **S2:** reduce the energy needed to hold the flap open by 30 %, by tuning the return spring and adding a low-current holding mode.
- **S3:** speed-scheduled controller gains with automatic tuning on the bench.

References

[1] The Race. What is DRS? Drag Reduction System explained. `https://www.the-race.com/formula-1/what-is-drs-drag-reduction-system-explained/`.

[2] D. De Cupis, H. de Carvalho Pinheiro, A. Ferraris, A. G. Airale, and M. Carello. Active aerodynamics design methodology for vehicle dynamics enhancement. In *Advances in Italian Mechanism Science (IFToMM ITALY 2020)*, volume 91 of *Mechanisms and Machine Science*, pages 777–785. Springer, 2021.

[3] S. Rijns, T.-R. Teschner, K. Blackburn, et al. Optimising vehicle performance with advanced active aerodynamic systems. *Vehicle System Dynamics*, 2025.

[4] J. Katz. *Race Car Aerodynamics: Designing for Speed*. Bentley Publishers, Cambridge, MA, 1995.

[5] International Organization for Standardization. ISO 26262:2018 – road vehicles – functional safety, 2018.

[6] A. Dell'Amico. Design and implementation of a hardware-in-the-loop air load simulation system for testing aerospace actuators. *Actuators*, 13(7):238, 2024.

[7] E. Sariyildiz, R. Oboe, and K. Ohnishi. Disturbance observer-based robust control and its applications: 35th anniversary overview. *IEEE Transactions on Industrial Electronics*, 67(3):2042–2053, 2020.

[8] FreeRTOS. FreeRTOS real-time operating system kernel. `https://www.freertos.org`.